HDOC Day-41

Paragraph Processing Using Strings, Functions, and Call by Reference in C

Aim

To write a menu-driven C program that reads two paragraphs and performs the required string-processing operations using separate user-defined functions, arrays, pointers, string.h, and ctype.h.

Algorithm

1. Read two paragraphs using readParagraph().
2. Display the menu and accept the user choice.
3. Find the longest and shortest words in each paragraph using strtok(), strlen(), and strcpy().
4. Remove repeated consecutive spaces with a character-by-character scan and isspace().
5. Convert each paragraph to title case using isalpha(), isspace(), tolower(), and toupper().
6. Find every distinct word frequency without considering letter case using strtok(), strcmp(), strcpy(), and tolower().
7. Display an error for an invalid choice.
8. Repeat until Exit is selected.

C Program

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
#define MAX 500
#define WORD 100
#define MAX_WORDS 100

void readParagraph(char p[], size_t size);
void findLongestAndShortestWords(const char p[], char longest[], char shortest[]);
void removeRepeatedSpaces(const char p[], char modified[]);
void convertToTitleCase(const char p[], char title[]);
void findWordFrequencies(const char p[]);

int main()
{
    char p1[MAX], p2[MAX], long1[WORD], short1[WORD];
    char long2[WORD], short2[WORD], modified1[MAX], modified2[MAX];
    char title1[MAX], title2[MAX];
    int choice;

    printf("Enter Paragraph 1:\n"); readParagraph(p1, MAX);
    printf("\nEnter Paragraph 2:\n"); readParagraph(p2, MAX);

    do {
        printf("\n----- PARAGRAPH OPERATIONS MENU -----\n");
        printf("1. Find Longest and Shortest Words\n");
        printf("2. Remove Repeated Consecutive Spaces\n");
        printf("3. Convert Paragraphs to Title Case\n");
        printf("4. Find Word Frequencies\n");
        printf("5. Exit\nEnter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
        case 1:
            findLongestAndShortestWords(p1,long1,short1);
            findLongestAndShortestWords(p2,long2,short2);
            printf("\nParagraph 1:\nLongest Word: %s\nShortest Word: %s\n",long1,short1);
            printf("\nParagraph 2:\nLongest Word: %s\nShortest Word: %s\n",long2,short2);
            break;
        case 2:
            removeRepeatedSpaces(p1,modified1); removeRepeatedSpaces(p2,modified2);
            printf("\nParagraph 1 after removing repeated spaces:\n%s\n",modified1);
            printf("\nParagraph 2 after removing repeated spaces:\n%s\n",modified2);
            break;
```

```
        case 3:
            convertToTitleCase(p1,title1); convertToTitleCase(p2,title2);
            printf("\nParagraph 1 in Title Case:\n%s\n",title1);
            printf("\nParagraph 2 in Title Case:\n%s\n",title2);
            break;
        case 4:
            printf("\nWord Frequencies - Paragraph 1:\n"); findWordFrequencies(p1);
            printf("\nWord Frequencies - Paragraph 2:\n"); findWordFrequencies(p2);
            break;
        case 5: printf("Exiting program...\n"); break;
        default: printf("Invalid choice! Please enter 1 to 5.\n");
        }
    } while (choice != 5);
    return 0;
}

void readParagraph(char p[], size_t size)
{
    fgets(p,size,stdin); p[strcspn(p,"\n")]='\0';
}

void findLongestAndShortestWords(const char p[], char longest[], char shortest[])
{
    char copy[MAX], *token; strcpy(copy,p); token=strtok(copy," \t");
    if(token==NULL){strcpy(longest,""); strcpy(shortest,""); return;}
    strcpy(longest,token); strcpy(shortest,token);
    while((token=strtok(NULL," \t"))!=NULL){
        if(strlen(token)>strlen(longest)) strcpy(longest,token);
        if(strlen(token)<strlen(shortest)) strcpy(shortest,token);
    }
}

void removeRepeatedSpaces(const char p[], char modified[])
{
    int i=0,j=0,prev=0;
    while(p[i]!='\0'){
        if(isspace((unsigned char)p[i])){if(!prev){modified[j++]=' '; prev=1;}}
        else {modified[j++]=p[i]; prev=0;} i++;
    }
    modified[j]='\0';
}

void convertToTitleCase(const char p[], char title[])
{
    int i=0,newWord=1;
    while(p[i]!='\0'){
        if(isspace((unsigned char)p[i])){title[i]=p[i]; newWord=1;}
        else if(isalpha((unsigned char)p[i])){
            title[i]=newWord?toupper((unsigned char)p[i]):tolower((unsigned char)p[i]); newWord=0;
        } else title[i]=p[i]; i++;
    }
    title[i]='\0';
}

void findWordFrequencies(const char p[])
{
    char copy[MAX],words[MAX_WORDS][WORD],*token; int freq[MAX_WORDS],count=0,i,found;
    strcpy(copy,p); token=strtok(copy," \t");
    while(token!=NULL){
        for(i=0;token[i]!='\0';i++) token[i]=tolower((unsigned char)token[i]);
        found=-1; for(i=0;i<count;i++) if(strcmp(words[i],token)==0){found=i;break;}
        if(found!=-1) freq[found]++; else {strcpy(words[count],token); freq[count++]=1;}
        token=strtok(NULL," \t");
    }
    for(i=0;i<count;i++) printf("%s : %d\n",words[i],freq[i]);
}
```

Test Case Table

Test	Operation	Example Input	Expected Result
1	Longest / Shortest	C programming is interesting	Longest=programming, Shortest=C
2	Repeated Spaces	C programming is fun	C programming is fun
3	Title Case	c programming is FUN	C Programming Is Fun
4	Word Frequency	C is fun and C is easy	c:2, is:2, others:1
5	Case-insensitive Frequency	Hello hello HELLO	hello:3

Test	Operation	Example Input	Expected Result
6	Two Paragraphs	Two different paragraphs	Processed separately
7	Invalid Choice	6	Invalid choice message
8	Exit	5	Program exits

Sample Output

```
Enter Paragraph 1:
C   programming is interesting and C is useful

Enter Paragraph 2:
hello HELLO world

Enter your choice: 1
Paragraph 1: Longest Word: programming, Shortest Word: C
Paragraph 2: Longest Word: hello, Shortest Word: hello

Enter your choice: 2
C programming is interesting and C is useful
hello HELLO world

Enter your choice: 3
C   Programming Is Interesting And C Is Useful
Hello Hello World

Enter your choice: 4
c : 2
programming : 1
is : 2
interesting : 1
and : 1
useful : 1
hello : 2
world : 1

Enter your choice: 5
Exiting program...
```

Result

The menu-driven C program was successfully implemented using separate user-defined functions, strings, arrays, pointers, string.h, and ctype.h. It reads two paragraphs, identifies longest and shortest words, removes repeated consecutive spaces, converts paragraphs to title case, and determines the frequency of every distinct word without considering letter case.